

QuickBite Sales & Menu Performance Analytics

Project Brief

For: Workshop build team

Purpose: Second AWS Glue capability track — sales/menu performance

1. Objective

Track 2 shows a different strength: single-source ingestion transformed into a proper dimensional model — a more “classic BI” pipeline that finance and menu/ops stakeholders.

2. The data story (presentation angle)

Framing: “Same menu, five branches — where is it actually working?”

QuickBite runs the same core menu across five branches in different cities, but performance isn't uniform — different products sell better in different markets, different channels (online, in-store, drive-thru) behave differently branch to branch. Today, comparing that across branches means manually opening the same export and re-slicing it by hand each time.

Three business questions this pipeline answers:

1. Branch performance — which branches are growing or declining in revenue, and is it price or volume driving the change?
2. Menu engineering — which products drive revenue vs. volume, and does the answer differ by branch (a product that's a star in London may be an underperformer in Berlin)?
3. Channel behavior — how does the purchase channel (online / in-store / drive-thru) and payment method vary by branch, and does channel mix correlate with revenue performance?

1. Branch performance — price vs. volume decomposition

Revenue change between periods splits cleanly into a price effect and a volume effect. This is standard price-volume-mix analysis:

sql

SQL

```
-- Step 1: monthly revenue, volume, and implied avg price per branch
WITH branch_monthly AS (
  SELECT branch_id, DATE_TRUNC('month', order_date) AS month,
         SUM(total_amount) AS revenue,
         SUM(quantity) AS total_qty,
         SUM(total_amount) / SUM(quantity) AS avg_price
  FROM fact_sales
  GROUP BY branch_id, DATE_TRUNC('month', order_date)
),
-- Step 2: month-over-month deltas
```

```

deltas AS (
  SELECT *,
    revenue - LAG(revenue) OVER (PARTITION BY branch_id ORDER BY month)
  AS revenue_change,
    total_qty - LAG(total_qty) OVER (PARTITION BY branch_id ORDER BY
month) AS qty_change,
    avg_price - LAG(avg_price) OVER (PARTITION BY branch_id ORDER BY
month) AS price_change,
    LAG(avg_price) OVER (PARTITION BY branch_id ORDER BY month) AS
prior_price
  FROM branch_monthly
)
-- Step 3: decompose
SELECT branch_id, month, revenue_change,
  qty_change * prior_price AS volume_effect,
  price_change * total_qty AS price_effect
FROM deltas

```

volume_effect = how much revenue moved because they sold more/less. **price_effect** = how much moved because price per unit changed. The two roughly sum to **revenue_change** (there's a small interaction term left over — fine to ignore at this scale, mention it exists if someone asks). This is the answer to "is it price or volume" — don't just report the growth %, report which lever moved it.

2. Menu engineering — with a caveat

Real menu engineering (BCG-style "stars/plow horses/puzzles/dogs") needs **profit margin**, which needs **cost data** — this dataset doesn't have cost. Say that plainly if the client asks; don't imply you're showing margin when you're showing revenue.

What you *can* build: a revenue-vs-volume quadrant per branch, and a cross-branch consistency check — which is actually the more interesting insight here anyway.

sql

```

SQL
-- quadrant per product per branch
WITH product_branch AS (
  SELECT branch_id, product_id,
    SUM(total_amount) AS revenue,
    SUM(quantity) AS volume,
    NTILE(2) OVER (PARTITION BY branch_id ORDER BY SUM(total_amount)) AS
revenue_tier,

```

```

        NTILE(2) OVER (PARTITION BY branch_id ORDER BY SUM(quantity)) AS
volume_tier
    FROM fact_sales
    GROUP BY branch_id, product_id
)
SELECT *,
    CASE WHEN revenue_tier = 2 AND volume_tier = 2 THEN 'core'
         WHEN revenue_tier = 2 AND volume_tier = 1 THEN 'premium/niche'
         WHEN revenue_tier = 1 AND volume_tier = 2 THEN 'value driver'
         ELSE 'underperformer' END AS quadrant
FROM product_branch

```

Then, the "star in London, underperformer in Berlin" check — rank each product within its branch, and find the products whose rank varies most across branches:

sql

```

SQL
WITH ranked AS (
    SELECT branch_id, product_id,
        RANK() OVER (PARTITION BY branch_id ORDER BY SUM(total_amount) DESC)
AS revenue_rank
    FROM fact_sales GROUP BY branch_id, product_id
)
SELECT product_id, STDDEV(revenue_rank) AS rank_volatility
FROM ranked
GROUP BY product_id
ORDER BY rank_volatility DESC

```

High **rank_volatility** = a product that performs very differently branch to branch — that's the specific finding worth surfacing, not just a static ranking.

3. Channel behavior

sql

```

SQL
-- channel mix and revenue share per branch
SELECT branch_id, purchase_type,
    COUNT(*) AS orders,
    SUM(total_amount) AS revenue,

```

```
SUM(total_amount) * 1.0 / SUM(SUM(total_amount)) OVER (PARTITION BY
branch_id) AS pct_of_branch_revenue
FROM fact_sales
GROUP BY branch_id, purchase_type
```

Then correlate channel mix against overall branch performance — pull each branch's online-revenue share and its growth rate (from Q1's output) into one row per branch, and check `CORR()`:

sql

```
SQL
SELECT CORR(pct_online_revenue, revenue_growth_rate) AS
channel_growth_correlation
FROM branch_summary
```

Caveat worth stating out loud in the demo: with only 5 branches, this correlation is directional at best, not statistically meaningful — say that rather than overselling a 5-point correlation as a finding. It's a fine illustration of the *method* you'd apply once the client has 50+ branches, which is the actual pitch.

Note: the original plan included a promotions/discount question — dropped, since the confirmed schema has no discount field. Don't promise that insight to the client on this dataset.

3. Data source (confirmed schema)

Source: Restaurant Sales Data (rohitgrewal) — Kaggle. Sales data of 5 branches of a restaurant in different cities, single CSV file.

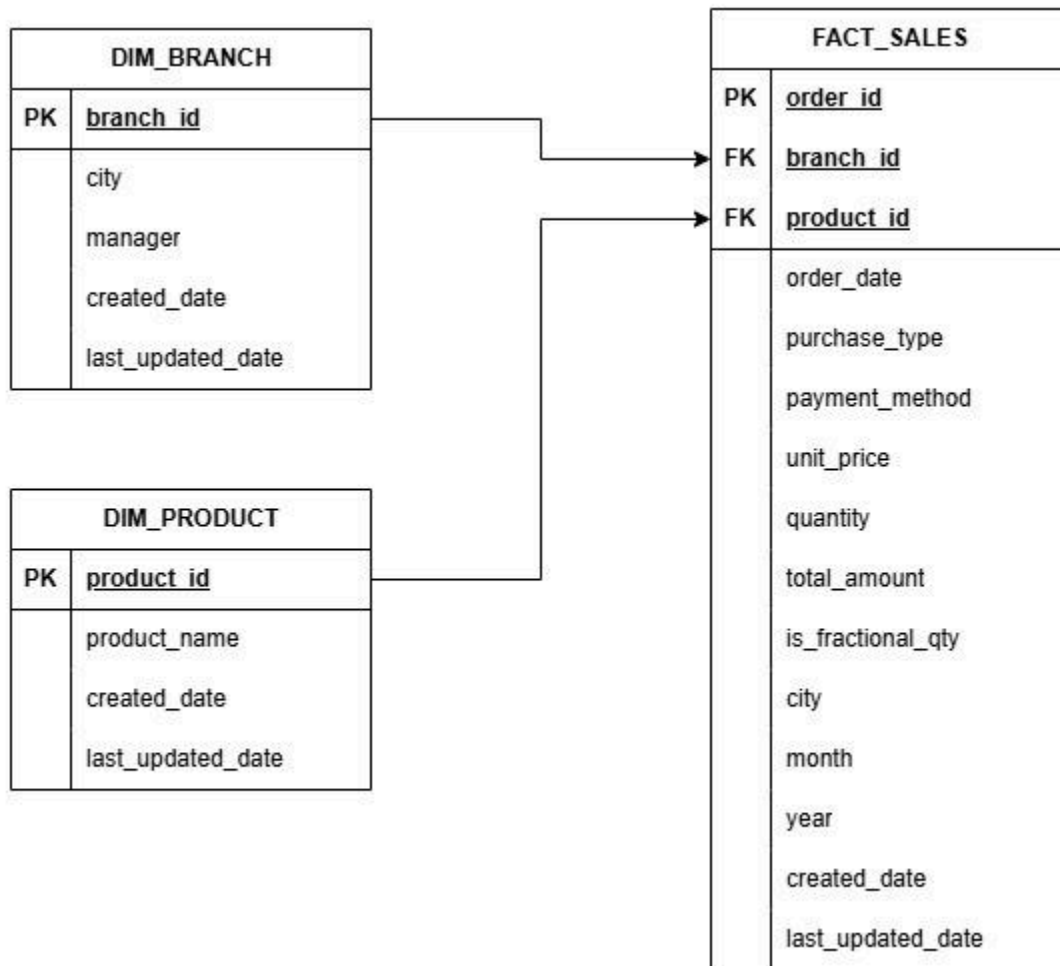
kaggle.com/datasets/rohitgrewal/restaurant-sales-data

Column	Type	Notes
Order ID	string	Unique identifier per transaction
Date	date	Order date
Product	string	5 known values: Burgers, Chicken Sandwich, Fries, Beverages, Sides & Other — acts as category, not individual SKU
Price	decimal	Per-unit price, observed range ~2.95–29.10. No currency column — assume single reporting currency unless proven otherwise
Quantity	decimal	Observed values include fractions (e.g. 200.4, 573.87, 745.76) — unusual for a unit count. Confirm whether this is a legitimate weighted/aggregated figure or a data quality artifact before treating it as ground truth
Purchase Type	string	Online / In-store / Drive-thru
Payment Method	string	Credit Card / Cash / Gift Card / other
Manager	string	Branch manager name — effectively 1:1 with branch
City	string	Branch location — 5 branches observed (London, Lisbon, Madrid, Berlin, +1)

Open item to resolve during build: the fractional Quantity values. Treat as a genuine data-quality question to investigate live in the workshop — it's a more authentic negative-test story than an invented one. See Section 8.

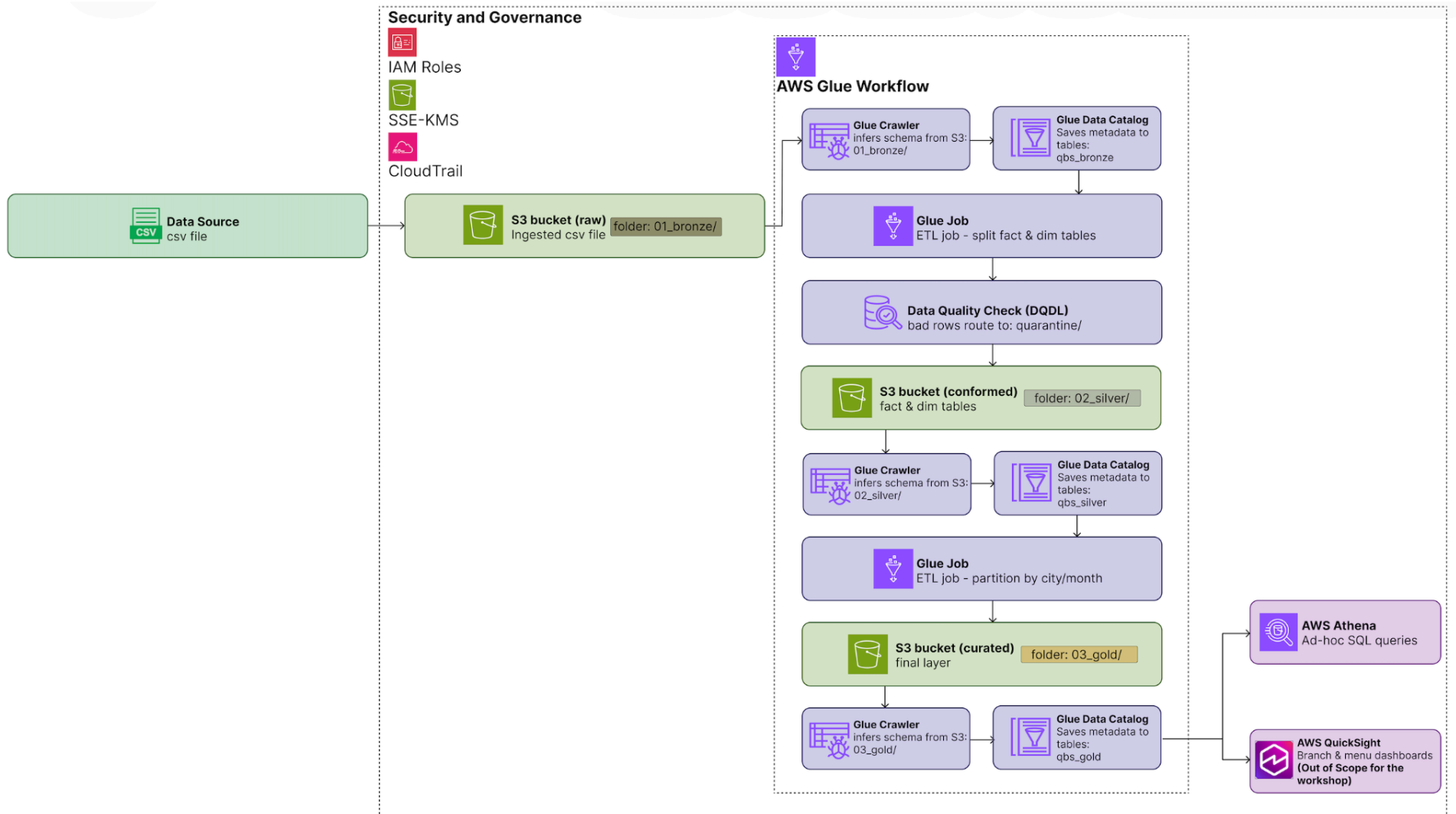
4. Data model — star schema

The raw CSV is flat (one row per transaction, city and manager repeated on every row). The ETL job normalizes it into a small star schema — this is also a good teaching moment for the newer teammates on why you normalize: City/Manager repeated across thousands of rows is redundant and error-prone to keep consistent; pulling it into a branch dimension fixes that at the source.



- fact_sales — order_id PK, branch_id FK, product_id FK, order_date, purchase_type, payment_type, payment_method, unit_price, quantity, total_amount (computed = unit_price x quantity), is_fractional_qty, city, month, year, created_date, last_updated_date
- dim_product — product_id PK, product_name, created_date, last_updated_date
- dim_branch — branch_id PK, city, manager, created_date, last_updated_date

5. Architecture



Data source → S3 raw layer (01_bronze) → AWS Glue workflow (scheduled trigger) → Glue Crawler (bronze) → Glue Data Catalog (qbs_bronze) → Glue ETL Job 1 (data cleaning - split fact & dim tables) → Data Quality Check (bad rows to: quarantine/) → S3 conformed layer (02_silver) → Glue Crawler (silver) → Glue Data Catalog (qbs_silver) → Glue ETL Job 2 (partition by city/month) → S3 curated layer (03_gold) → Glue Crawler (gold) → Glue Data Catalog (qbs_gold) → Athena / QuickSight

6. Build tasks

#	Task	Depends on	Assigned to	Status	Remarks
1	Setup S3 Bucket and upload Sales-Data-Analysis.csv to s3://.../sales_raw/	—	Hazel M...	Done ▾	
2	Create Documentation: Data model		Neil Tim...	Done ▾	Completed data model (fact_sales + 2 dim tables) with s3 partitioning strategy. For peer review.
3	Create Documentation: Architecture Diagram		Franch...	Done ▾	Adjusted architecture diagram according to planned flow
4	Create Crawler and AWS Glue Data Catalog for each layer in the medallion architecture	Task 1	Hazel M...	Done ▾	
5	Investigate the fractional Quantity pattern — decide handling before building the ETL logic around it	Task 4	Neil Tim...	Done ▾	Retain Decimal(10, 2) precision and add is_fractional_qty flag in Gold layer to preserve revenue integrity without dropping data.
6	ETL job bronze-to-silver: implement cleansing, DLQ routing, split into fact_sales + dim_product + dim_branch, partition fact by city/month	Task 1,2, 4, & 5	Hazel M...	Done ▾	
7	ETL job silver-to-gold: build aggregated business metrics table/s	Task 6	Neil Tim...	Done ▾	
8	DQDL ruleset (Section 8)		Franch...	Done ▾	Named DQDL ruleset as: qbs_dqdl_ruleset and placed in: qbs_bronze database > sales_raw table. Other rulesets (PySpark) are inside ETL job
9	Create test dataset & Perform negative testing on ETL pipeline		Franch...	Done ▾	Uploaded test dataset named: testing-dqdl.csv and placed in: qbs-workshop-bucket > 01_bronze/ > test_dataset/ to perform the DQDL testing.
10	Design AWS Glue Studio Visual ETL pipeline for demo	Task 6 & 7		Done ▾	
11	Athena query answering one of the 3 business questions	Task 7		Done ▾	

7. Resiliency, cost, and security

- Job bookmarks enabled on the ETL job
- Flex execution class — this pipeline has no urgency requirement, a strong Flex candidate to contrast against Track 1's more time-sensitive jobs
- DQDL rules: Order ID not null, Price > 0, Quantity > 0, City in known branch list
- IAM role scoped to raw-read / curated-write only
- SSE-KMS on both buckets
- S3 Data Security

8. Negative testing plan

#	Test case	Inject	Expected behavior
1	Negative or zero quantity	Quantity = -5	DQDL / PySpark validation rejects the row and sends it to quarantine
2	Missing price	Null Price	`unit_price` becomes `null`; the row is quarantined and the original `Price` value is retained
3	Fractional quantity anomaly	A Quantity like 573.87 (already present in real data)	The row remains valid and is written to reporting data with `is_fractional_qty = true`, allowing it to be identified for review
4	Duplicate Order ID	Same Order ID twice	Invalid rows are quarantined first, among valid duplicates, the latest `date` is kept
5	Unrecognized product	A Product value outside the 5 known categories	The row is quarantined and does not create a `dim_product` record. For valid new products, they are quarantined until reviewed and added to the approved product list.
6	Unknown branch/city	A City not among the 5 known branches	The row is quarantined and does not create a `dim_branch` record
7	Date format inconsistency	A row with a different date format than the rest	Parsing fails gracefully to errors/, not a silent bad date

9. Scale-up scenario

- Daily ingestion of POS data for analysis

10. Success criteria

- ☐ Fractional Quantity pattern investigated and documented, not ignored
- ☐ fact_sales + 2 dimension tables visible and correctly typed in the Glue Data Catalog
- ☐ At least 2 negative test cases run and documented
- ☐ Athena query returns a real answer to one of the 3 business questions
- ☐ Every teammate can explain why the star schema was chosen over the flat source

11. AWS Resources Used

#	Resource Name	Service	Status	Notes
1	qbs-workshop-bucket	S3	Available ▾	
2	qbs_csv_classifier	Glue - Classifier	Available ▾	
3	qbs_bronze	Glue - Database	Available ▾	Separated the database in AWS Glue Data Catalog to ensure data governance. This is to enforce layer-specific access for tools like the Amazon Quicksight.
4	qbs_silver	Glue - Database	Available ▾	
5	qbs_gold	Glue - Database	Available ▾	
6	qbs_quarantine	Glue - Database	Available ▾	
7	qbs_bronze_crawler	Glue - Crawler	On-Demand ▾	

#	Resource Name	Service	Status	Notes
8	qbs_AWSGlueServiceRoleAccess	IAM Role	Available ▾	Created a new IAM role: `qbs_AWSGlueServiceRoleAccess` with the policy: `QBS_GlueKMSAccess` which uses the AWS managed key for SSE-KMS encryption.
9	qbs_bronze-to-silver	Glue - ETL Job	Available ▾	
10	qbs_silver-to-gold	Glue - ETL Job	Available ▾	
11	qbs_workgroup	Athena - Workgroup	Available ▾	Currently set to 50 GB for the data scan limit
12	qbs_etl_pipeline	Glue - Visual ETL	Available ▾	
13	qbs_quarantine_crawler	Glue - Crawler	On-Demand ▾	
14	qbs_dqdl_ruleset	Glue - DQDL	Available ▾	Data Quality ruleset that directs failed results to quarantine